International Islamic University Chittagong

Department of Computer Science and Engineering

B. Sc. In CSE Midterm Examination, Spring 2023

Course Code: CSE 2421 Course Title: Computer Algorithms

Total marks: 30 *Time: 90 minutes*

[Answer all the questions; In some questions, there might be options; Figures in the right hand margin indicate full marks.]

2c

1. a) Find the asymptotic upper bound of the following recurrence by using master method: 4
   $T(n) = 3T(n/3) + f(n)$
   where (I) $f(n) = n^2$ and (II) $f(n) = \lg n$.

   b) Find the time complexity of the following function. 3

```
void Duffy (int n)
{
  for (int i = 1; i*i <= n; i++)
    for (int j = 1; j <= i; j = j*2)
      k = k + 1;
}
```

   How the time complexity will be changed if i*i is replaced with i?

   c) Show that the asymptotic lower bound of any polynomial of degree n can be expressed as $\Omega(x^n)$. 3

2. a) Find the time complexity of the following function? 4

```
Raise (x, n)
  if x = 0
    return 1
  p = floor(n/2)
  if n is odd      return x * Raise(x, p) * Raise(x, p)
  if n is even     return Raise(x, p) * Raise(x, p)
```

   Can you specify a way to improve the performance of the function?

   b) Show using recursion tree how the worst case time complexity of quicksort algorithm becomes $O(n^2)$. 3

   c) Construct a Heap from the following array and show each step. 3
   $A = (5, 9, 3, 14, 7, 12, 11, 8, 4, 10, 6)$.

   OR

   Show how you can sort the numbers from the following Heap A. Illustrate each step.
   $A = (10, 9, 8, 7, 6, 5, 4, 3, 2, 1)$.

3. a) Find an optimal parenthesization of a matrix-chain product whose sequence of dimensions is 4
   $(5, 5, 6, 6, 8)$.

   OR

   Find a Longest Common Sequence of the following two sequences using dynamic programming and show the steps.
   $X = (0, 1, 1, 0, 1, 0, 1)$
   $Y = (1, 0, 0, 0, 1, 1)$

   b) What is optimal substructure? Show that the matrix-chain-multiplication problem has this property. 3

   OR

   What is overlapping subproblems? Show that the longest common subsequence problem has this property.

   c) Show the operation of the Counting-sort on the following array. 3
   $A = (6, 4, 8, 4, 6, 2, 6, 4, 8, 2, 7)$

Spring 23

(19)

Find the asymptotic upper bound of the following recurrence by using master method :

$$T(n) = 3T(n/3) + f(n)$$

where, (i) $f(n) = n^2$ (ii) $f(n) = \log n$

(i) $T(n) = 3T(n/3) + n^2$

$a = 3$

$b = 3$

$k = 2$

$p = 0$

$$T(\theta) = \theta(n^k \log^p n)$$

$$= \theta(n^2 \log^0 n)$$

$$= \theta(n^2 \log^{0-1} n)$$

$$= \theta(n^2)$$

$a < b^k$

$3 < 3^2$

$3 < 9$

(11) $T(n)=3T(n/3)+\log n$

$T(n)=aT(n/b)+f(n)$

$f(n)=\log(n)$

$n^c = n^{\log_3^3}$

$a=3$

$b=3$

$f(n)=\log n$

$n^c \Rightarrow c=\log 3(3)$

$T\Theta\xi=\Theta(n^{\log_b^a})$

$=\Theta(n^{\log_3^3})$

$=\Theta(n^1)$

$=\Theta(n)$

Ans. to 1 (b)

Part 1: The outer loop of given function runs till $9*9 <= n$. Thus, 9 goes upto the Square root of n.

$\therefore$ The time complexity of outer loop $= O(\sqrt{n})$

The inner loop runs till $j <= 9$ and in each iteration $j = j*2$.

$\therefore$ The time complexity of inner loop. 9s the sum of all possible values of $9 : (9 = 1, 2, 3 \ldots \sqrt{n})$

$$\therefore T(n) = O(\log 1) + O(\log 2) + O(\log 3) + \ldots + O(\log \sqrt{n})$$

$$= O(\log(1 + 2 + 3 + \ldots \sqrt{n}))$$

$$= O\left(\log \frac{\sqrt{n}(\sqrt{n}+1)}{2}\right)$$

$$= O(\log(n + \sqrt{n}))$$

$$\therefore T(n) = \theta(\log n)$$

Part 2: When q*q is replaced by q, the outer loop runs till q<=n. Thus q goes upto n. The inner loop remains same.

$\therefore T(n) = O(\log 1) + O(\log 2) + O(\log 3) + \ldots + O(\log n)$

$= O(\log(1+2+3+\ldots+n))$

$= O\left(\log \frac{n(n+1)}{2}\right)$

$= O(\log(n^2+n))$

$= O(\log n)$.

∴ After replacing q*q by q in the function, the time complexity remains same.

Ans. to 1(c)

We know that a function $f(x)$ is said to be $\Omega(x^n)$ if there exist positive constants c and x_0 such that for all x greater or equal to x_0, the function $f(x)$ is greater than or equal to ~~$c \cdot x^n$~~ cx^n.

Let, a polynomial function,

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots\ldots + a_1 x + a_0.$$

where $a_n \neq 0$.

To establish the lower bound, we can focus on the term with the highest degree, $a_n x^n$.

Let, $c = |a_n|$ and $x_0 = 1$.

For $x \geq 1$, each term in $a_k x^k$ in the ~~polyno~~ polynomial $(0 \leq k \leq n-1)$ will be dominated by $a_n x^n$, because x^n grows faster

than x^k. So, we can ignore those lower degree terms.

Now, for $x \geq 1$:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \geq |a_n| x$$

As $|a_n| = c$ and $x_0 = 1$ we have established that for all $x \geq x_0$,

$$f(x) \geq c x^n.$$

This satisfies the definition of $\Omega(x^n)$.

So, it is shown that the asymptotic lower bound of any polynomial of degree n can be expressed as $\Omega(x^n)$.

2_a

a) Find the time complexity of the following function and specify a way to improve the performance of the function.

⇒ Raise (x, n)

if x = 0

return 1

P = floor (n/2)

if n is odd return x * Raise(x,P) + Raise(x,P)

if n is even return Raise (x,P) + Raise (x,P)

The time complexity of this function is O(logn) cause it follows a pattern that reduces the problem size by half in each step.

Mathematically, the number of depth of recursion required to reach a base case (like n=0) is proportional to $\log_2(n)$, where n is the exponent.

A way to improve the performance of the function:

```
int Raise (int x , int n ) {
    if (n == 0 ) {
        return 1 ;
    }
int  P = Raise (x, n/2);

if (n % 2 == 0 ){
        return  P * P ;
}else{
        return  x * P * P ;
    }
}
```

The given recursive approach recalculates the power for both even and odd exponents, resulting in redundant computations. But exponentiation by squaring reduces the number of recursive calls by dividing the exponent by 2 at each step.

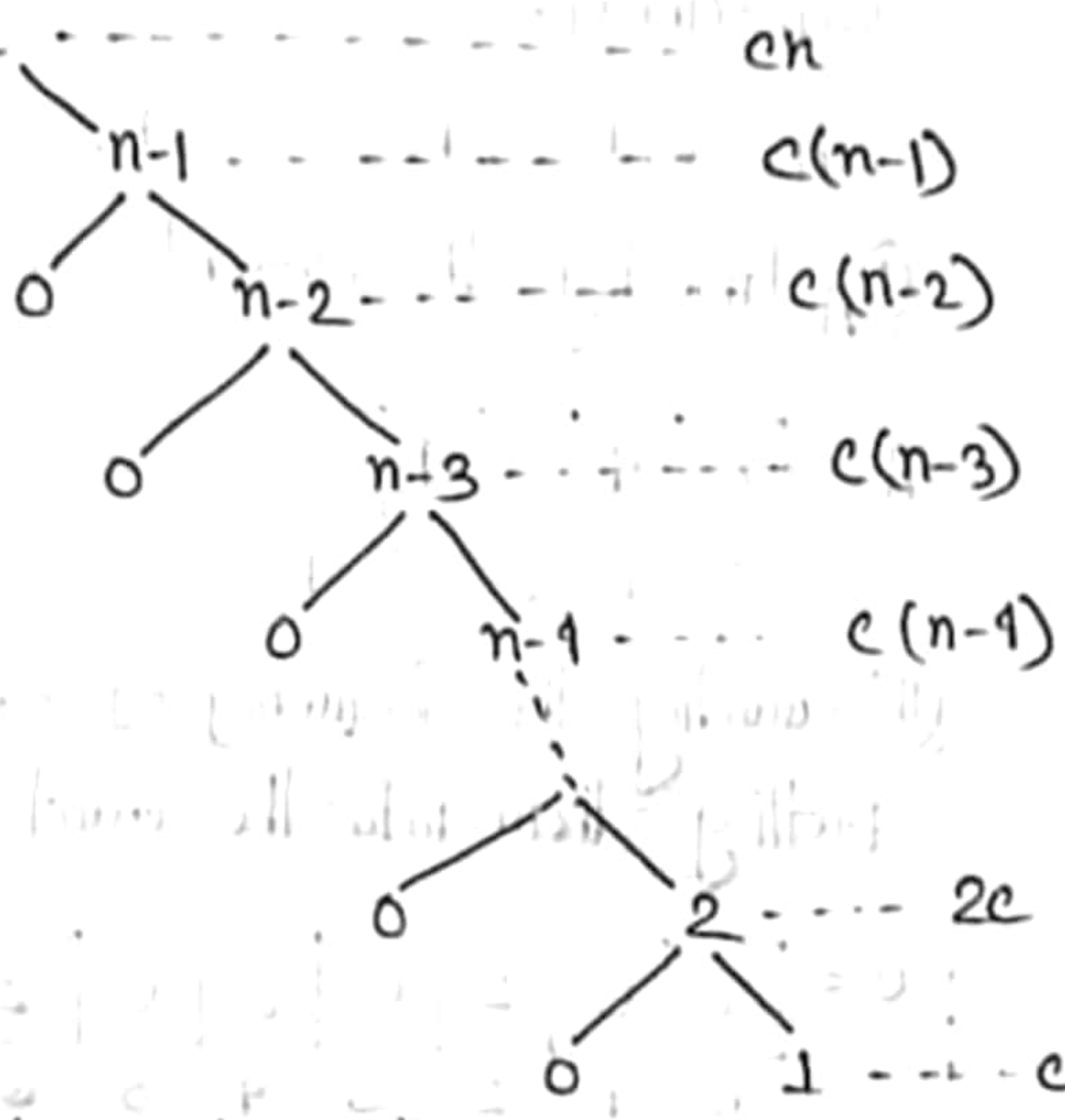
2(b)

The worst case scenario for the Quick-Sort algorithm occurs when the pivot chosen for partitioning the array is consistently either the smallest or the largest element in each partition. This leads to unbalanced partition, where one subarray has 'n-1' element and the other subarray is empty. Worst-case always occurs in the case of sorted, reverse sorted and on all elements are same. Let, $T(n)$ is the time complexity of quicksort for input size 'n', Input size of left subarray is 'i' and size of right subarray = n-i-1. For the worst case of quick sort, we put, $i = n-1$, in the recurrence relation and we get,

$$T(n) = T(n-1) + T(0) + cn$$

$$= T(n-1) + cn$$

The recursion tree : (ascending)



In this method, we draw the recursion tree and sum the partitioning cost for each level,

$$= cn + c(n-1) + c(n-2) + c(n-3) + \cdots + 2c + c$$

$$= c(n + n-1 + n-2 + n-3 + \cdots + 2 + 1)$$

$$= c(n(n+1)/2)$$

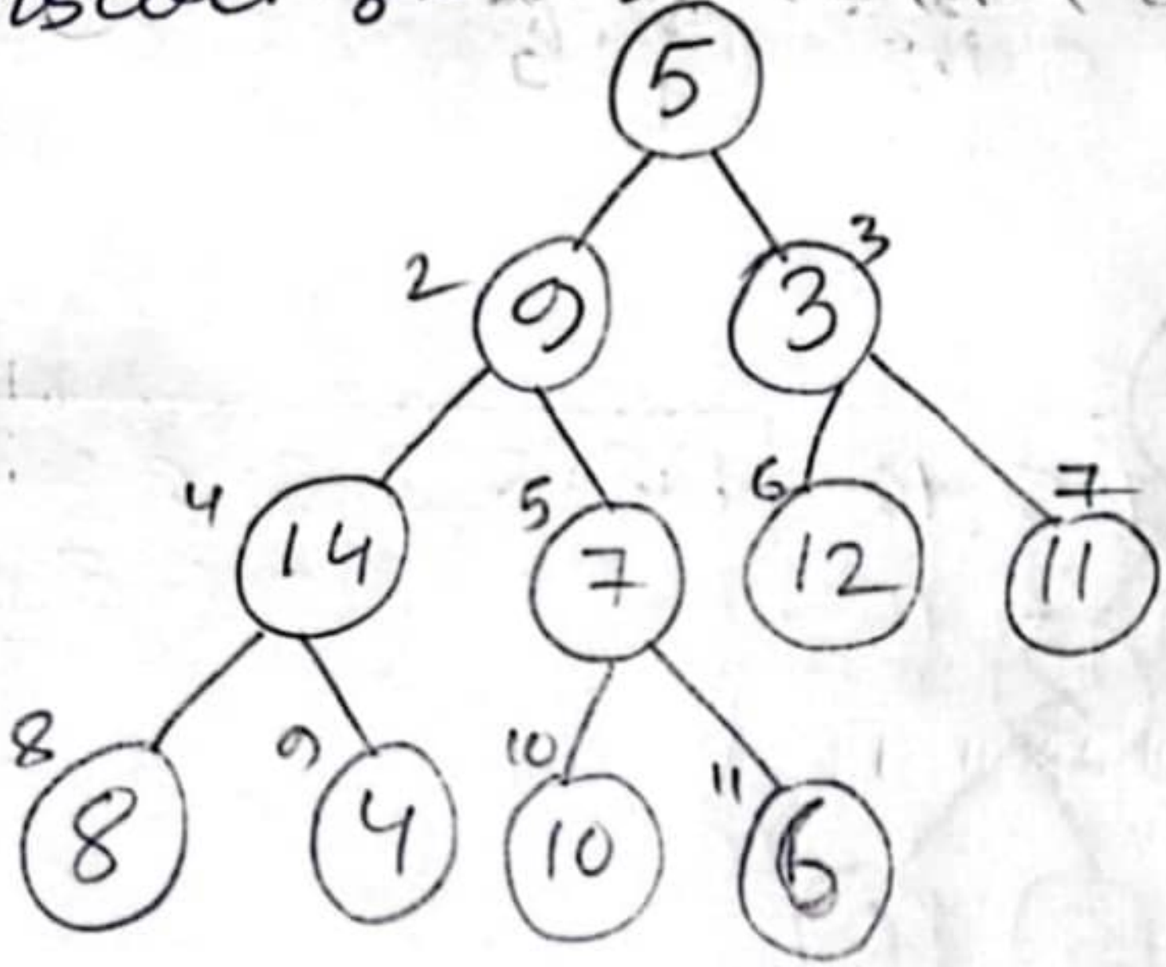
$$= O(n^2)$$

So, worst case time complexity for quicksort is $O(n^2)$.

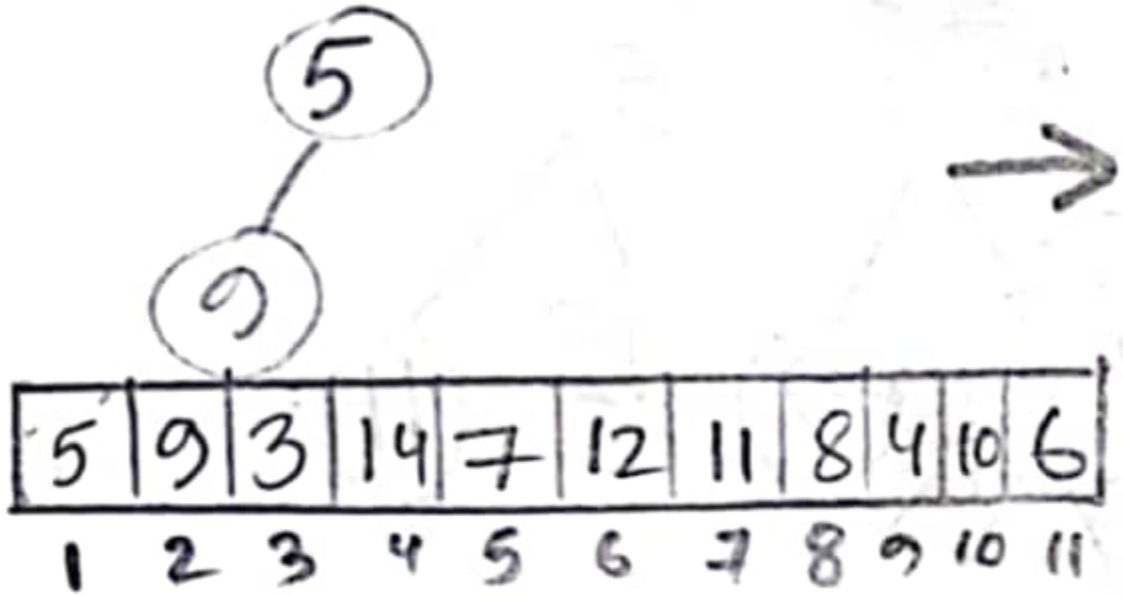
2(c) Construct a Heap from the following array and show each step

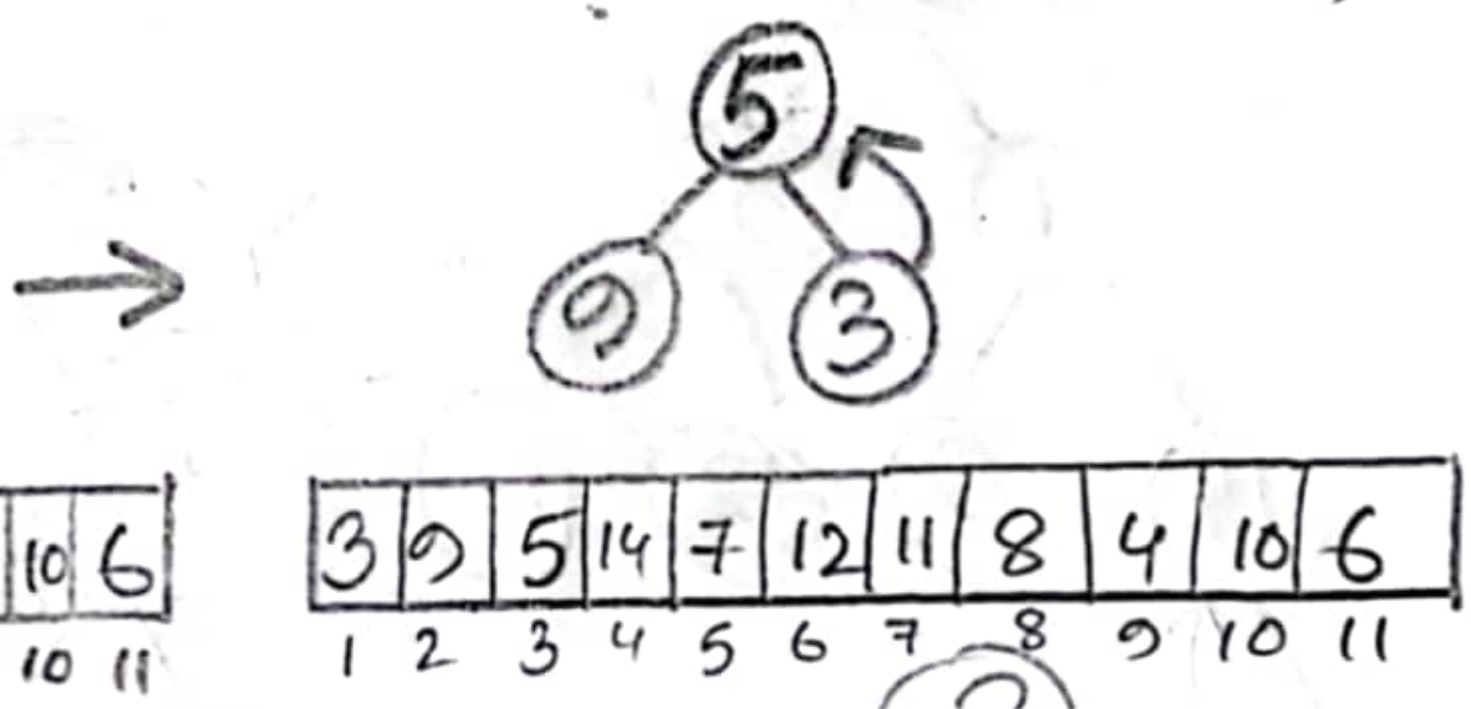
$A = \{5, 9, 3, 14, 7, 12, 11, 8, 4, 10, 6\}$

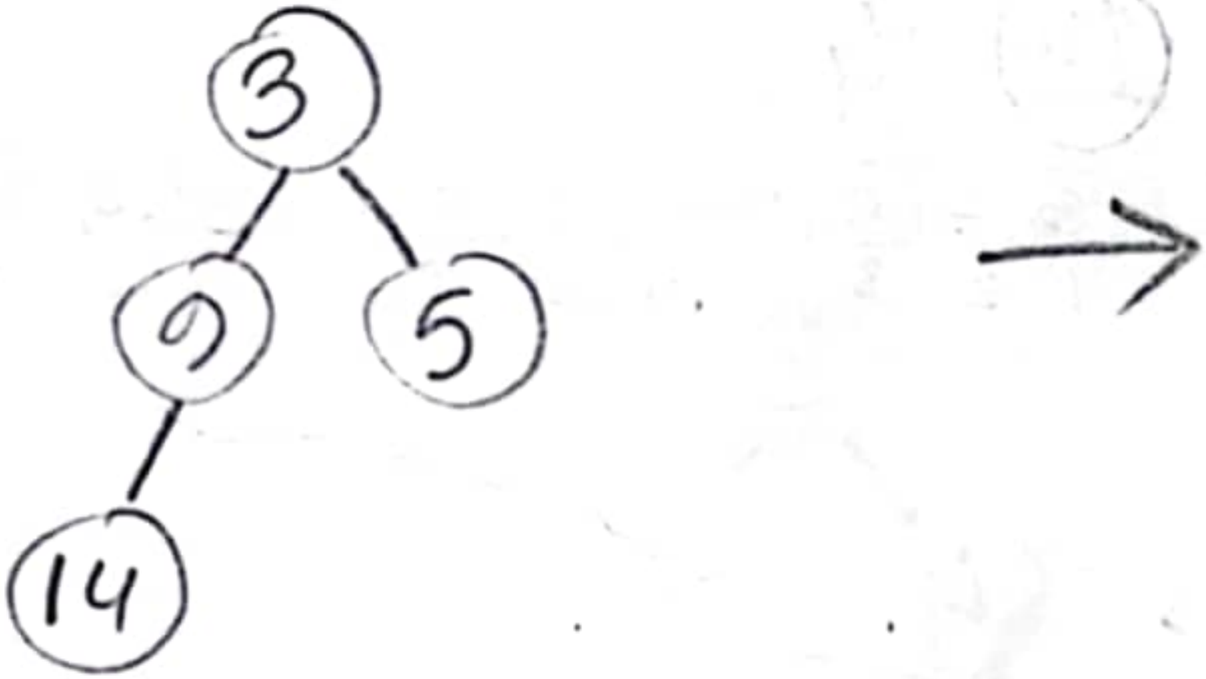
Answer:

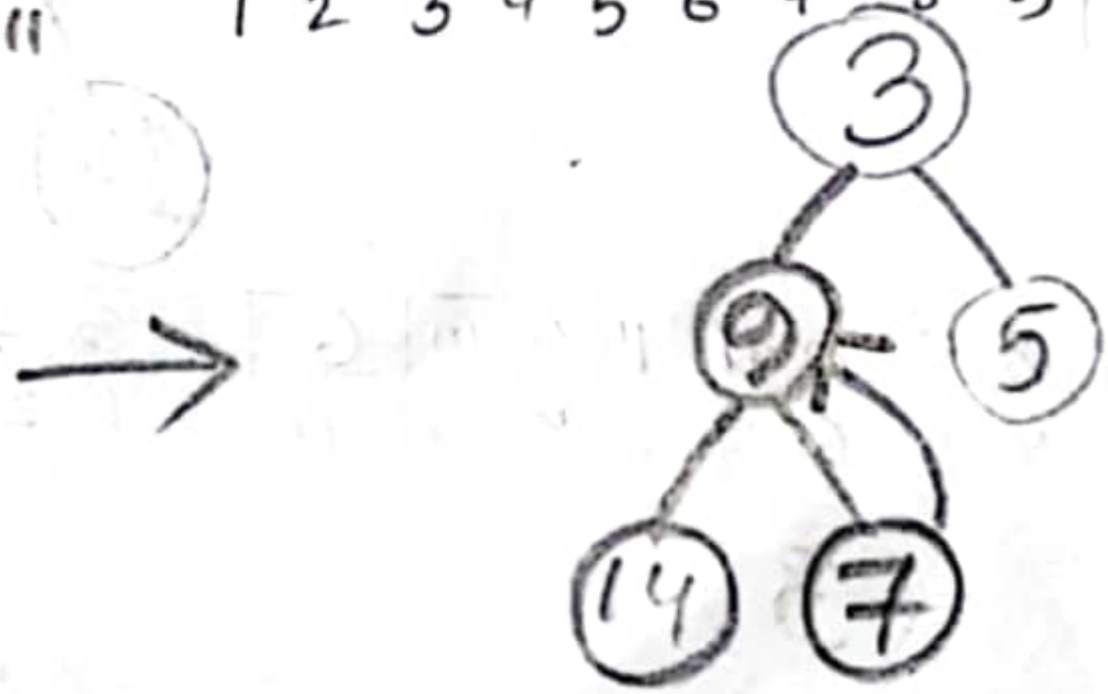


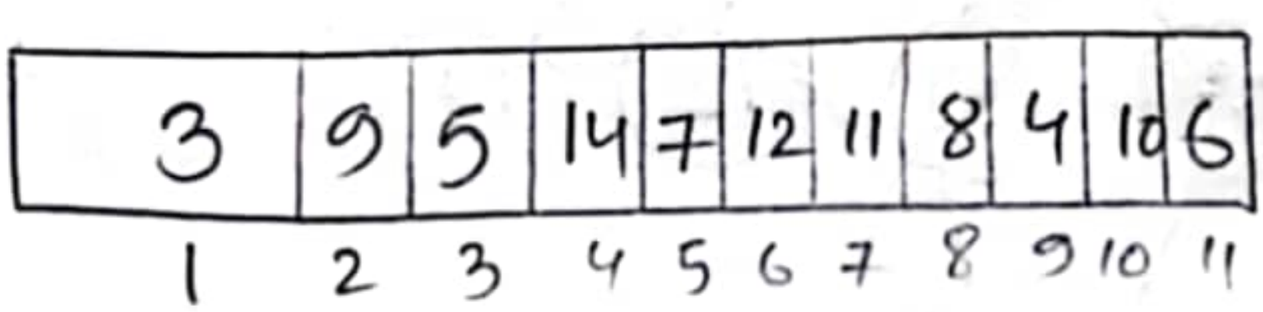
Min Heap (parent node < child node)

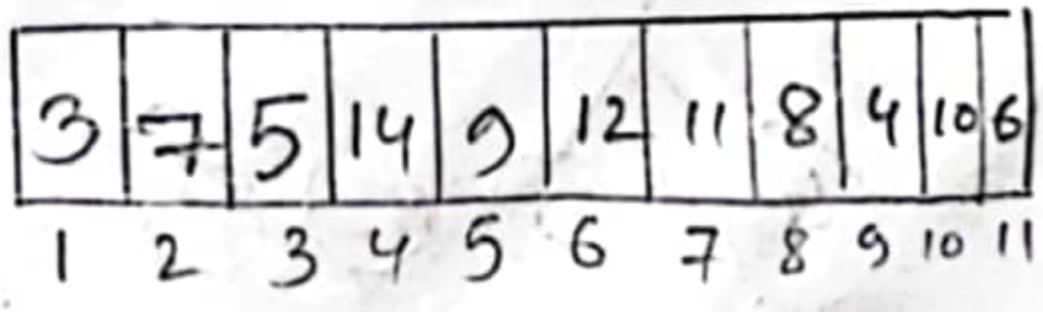


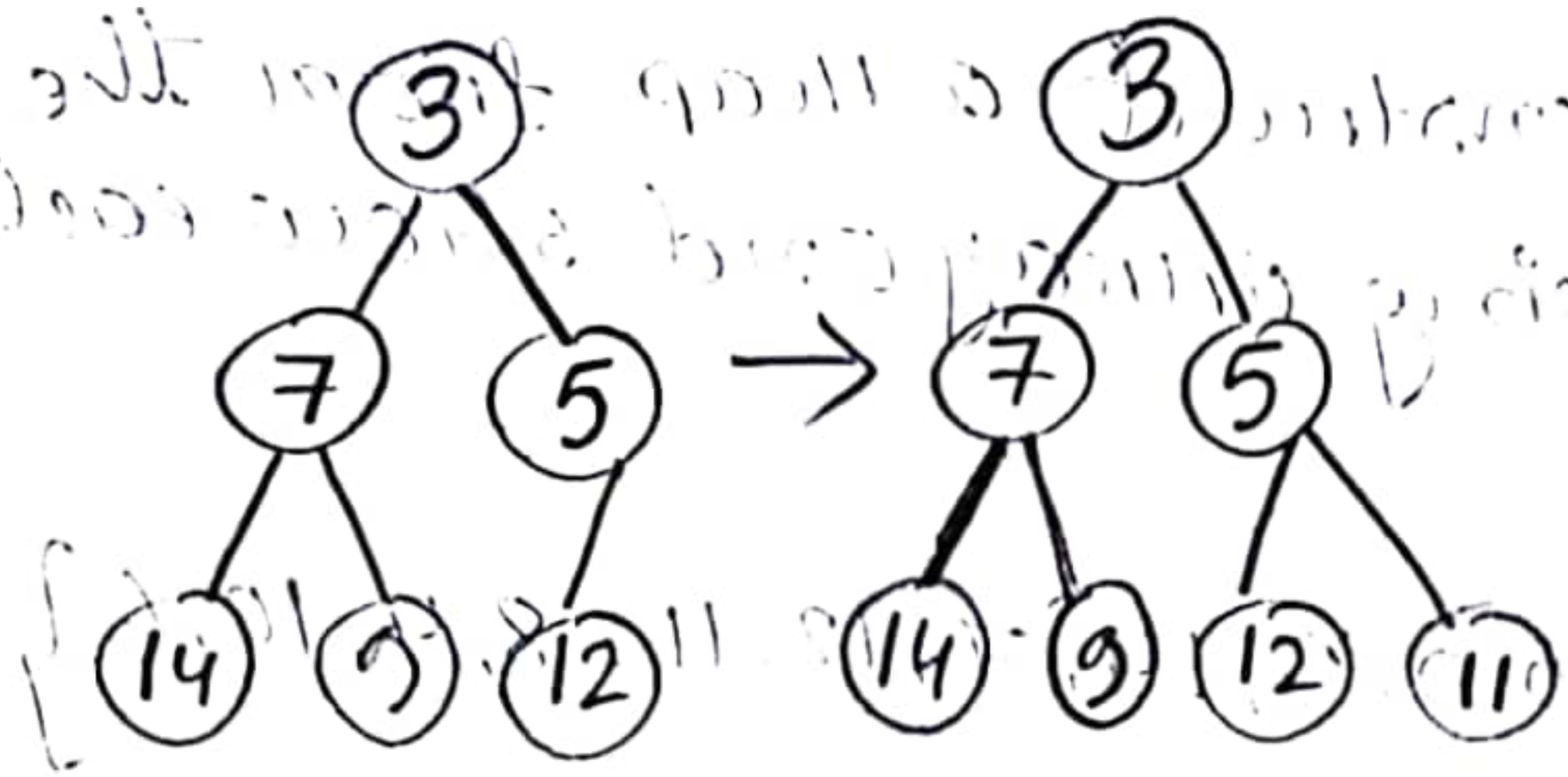






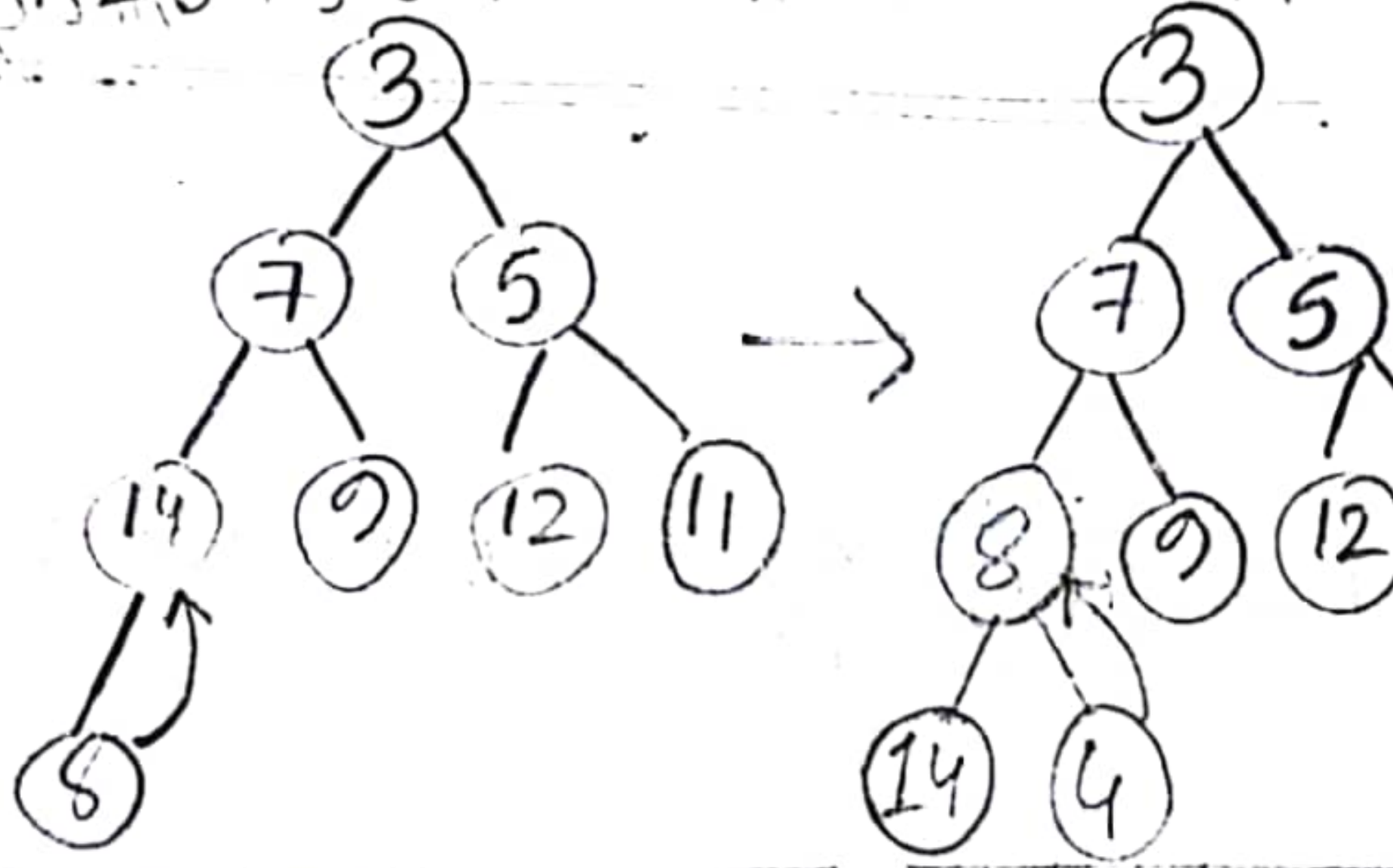






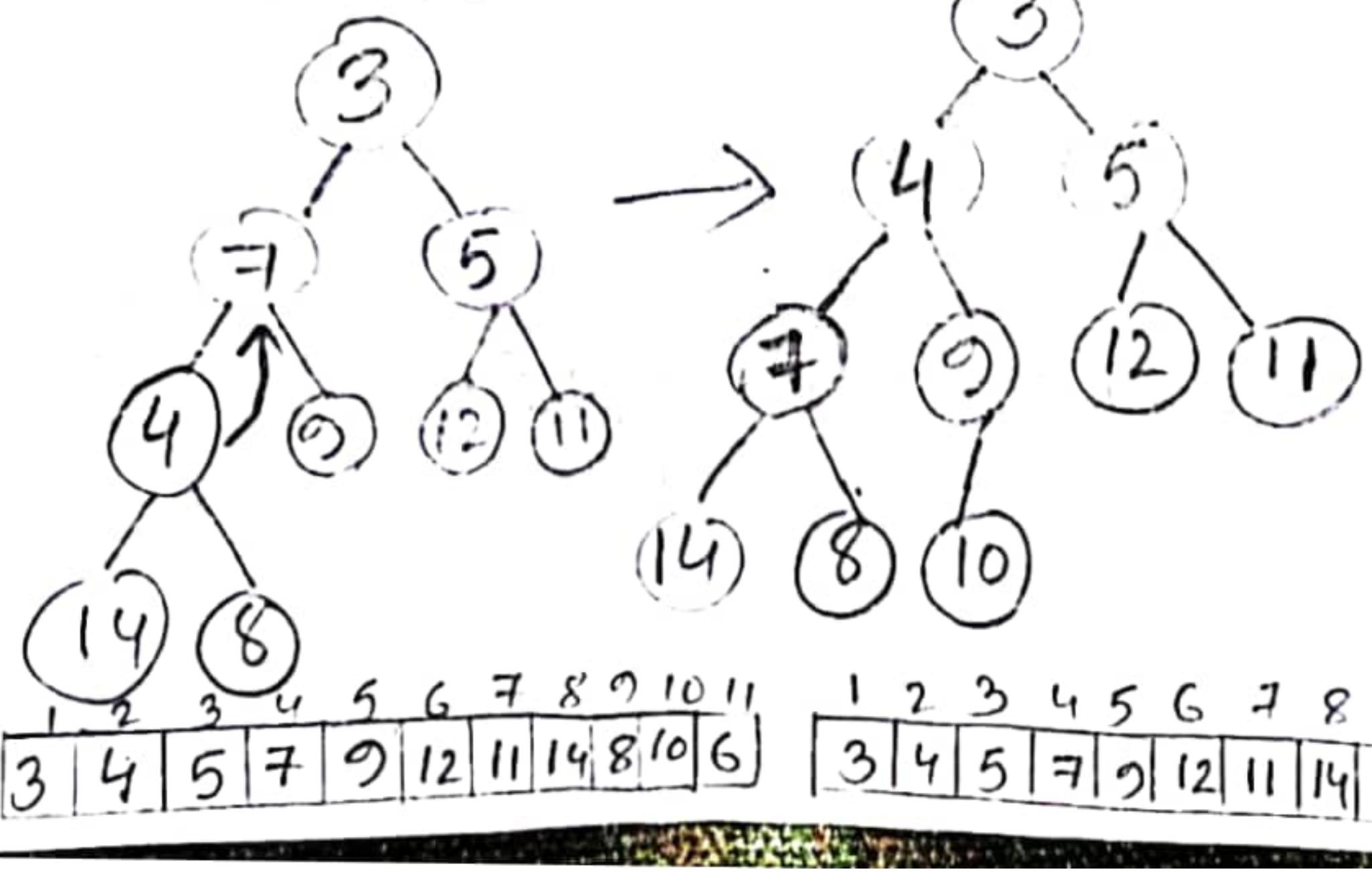
3	7	5	14	9	12	11	8	4	10	6
1	2	3	4	5	6	7	8	9	10	11

3	7	5	14	9	12	11	8	4	10	6
1	2	3	4	5	6	7	8	9	10	11



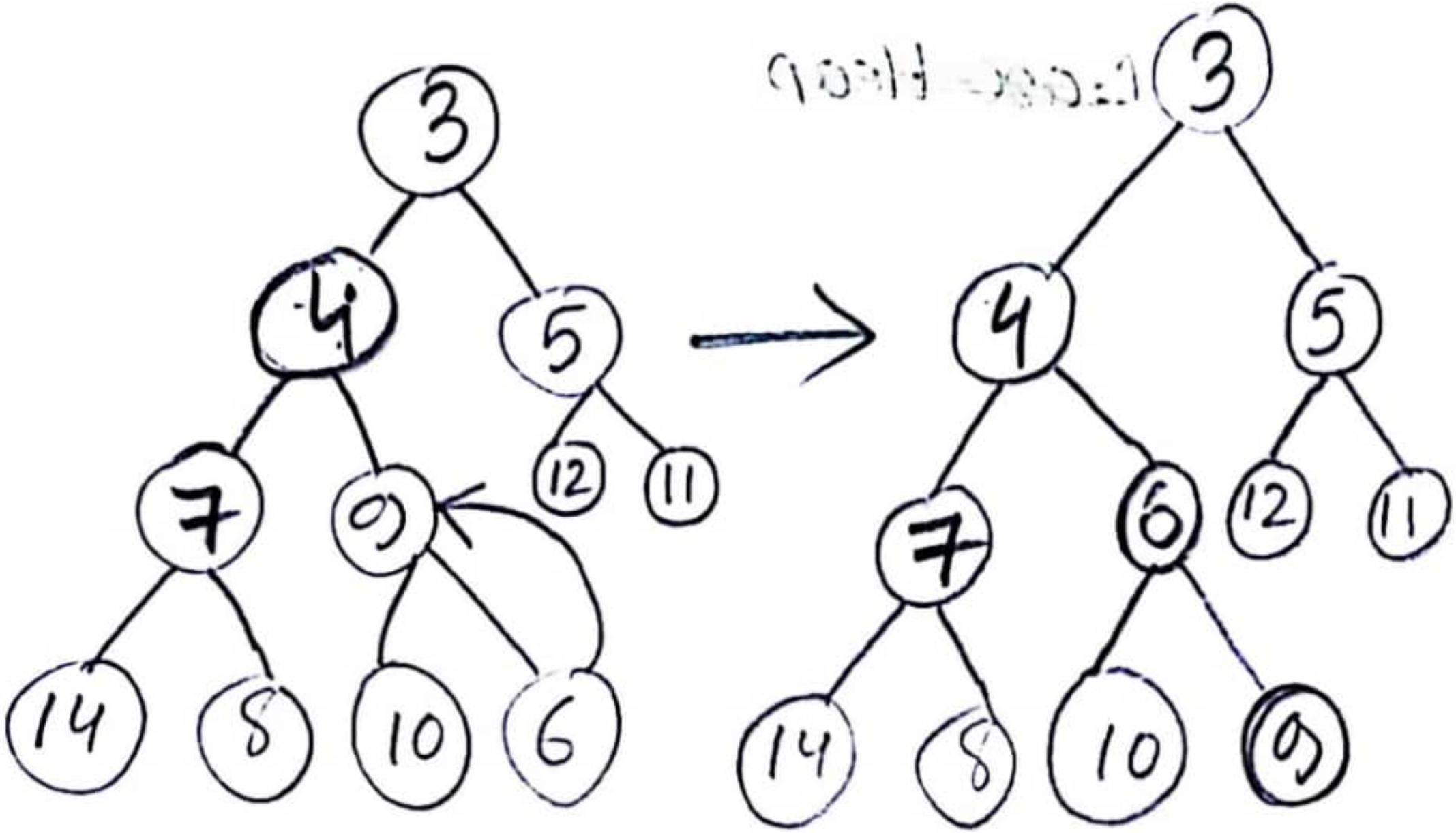
3	7	5	8	9	12	11	14	4	10	6
1	2	3	4	5	6	7	8	9	10	11

3	7	5	4	9	12	11	14	8	10	6
1	2	3	4	5	6	7	8	9	10	11



3	4	5	7	9	12	11	14	8	10	6
1	2	3	4	5	6	7	8	9	10	11

3	4	5	7	9	12	11	14	8	10	6
1	2	3	4	5	6	7	8	9	10	11

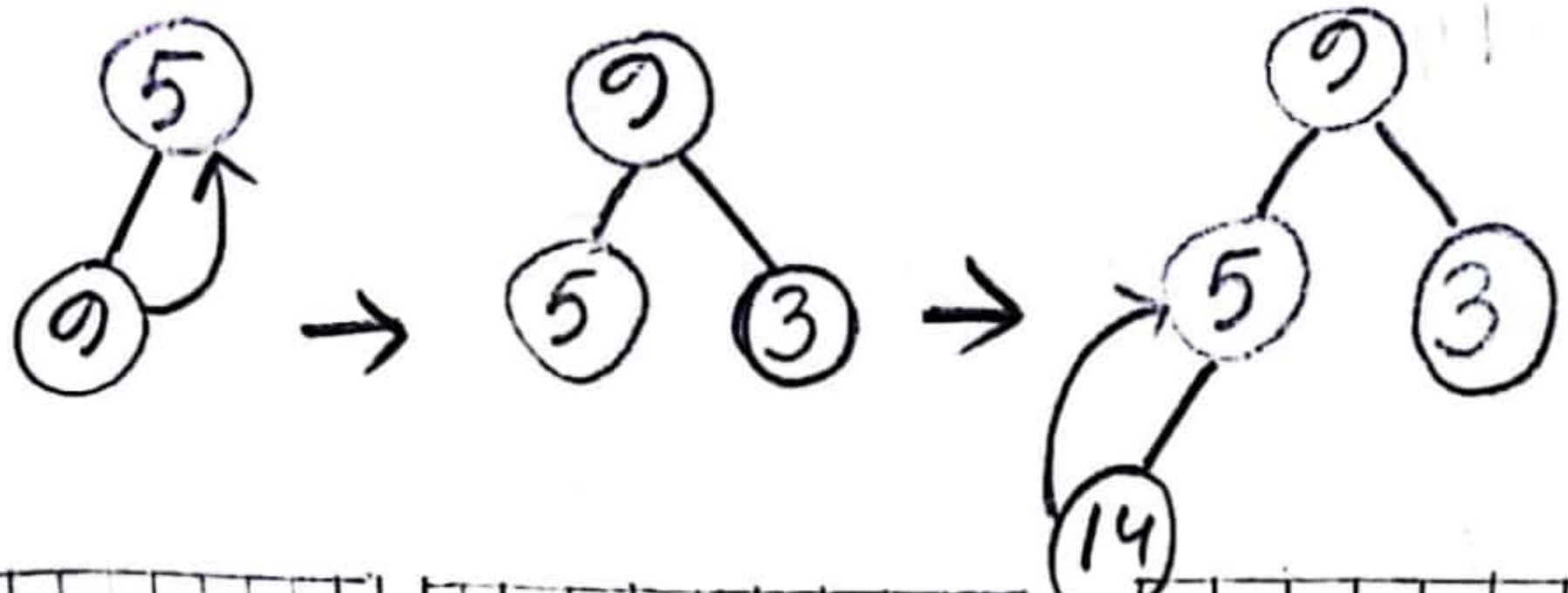


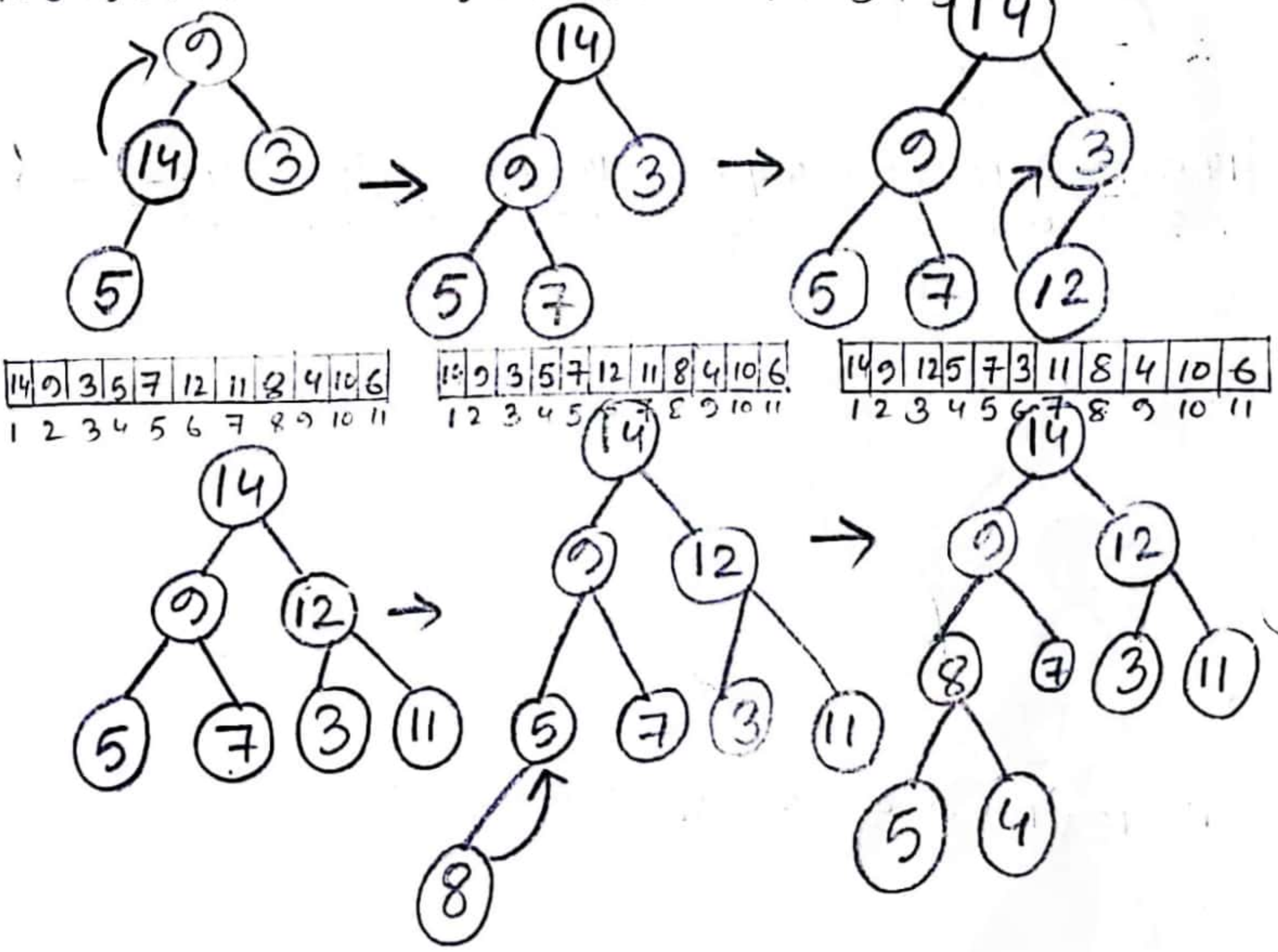
1	2	3	4	5	6	7	8	9	10	11
3	4	5	7	6	12	11	14	8	10	9

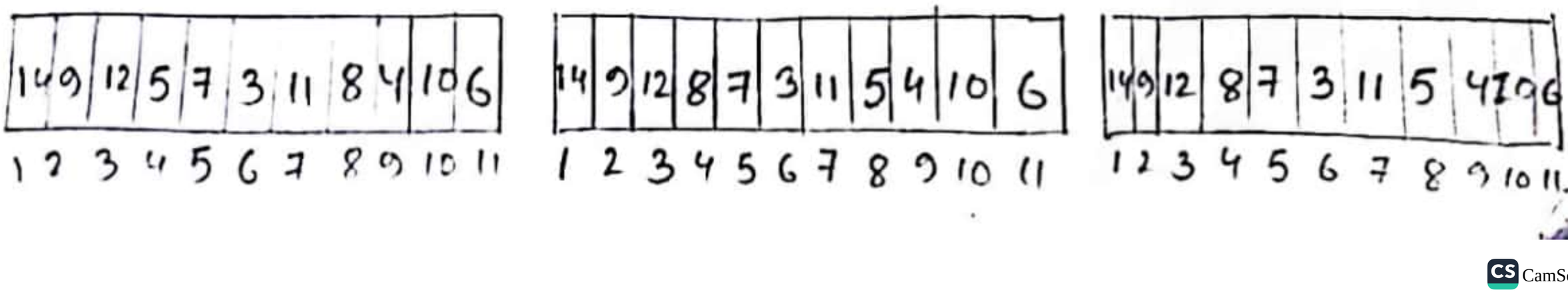
1	2	3	4	5	6	7	8	9	10	11
3	4	5	7	6	12	11	14	8	10	9

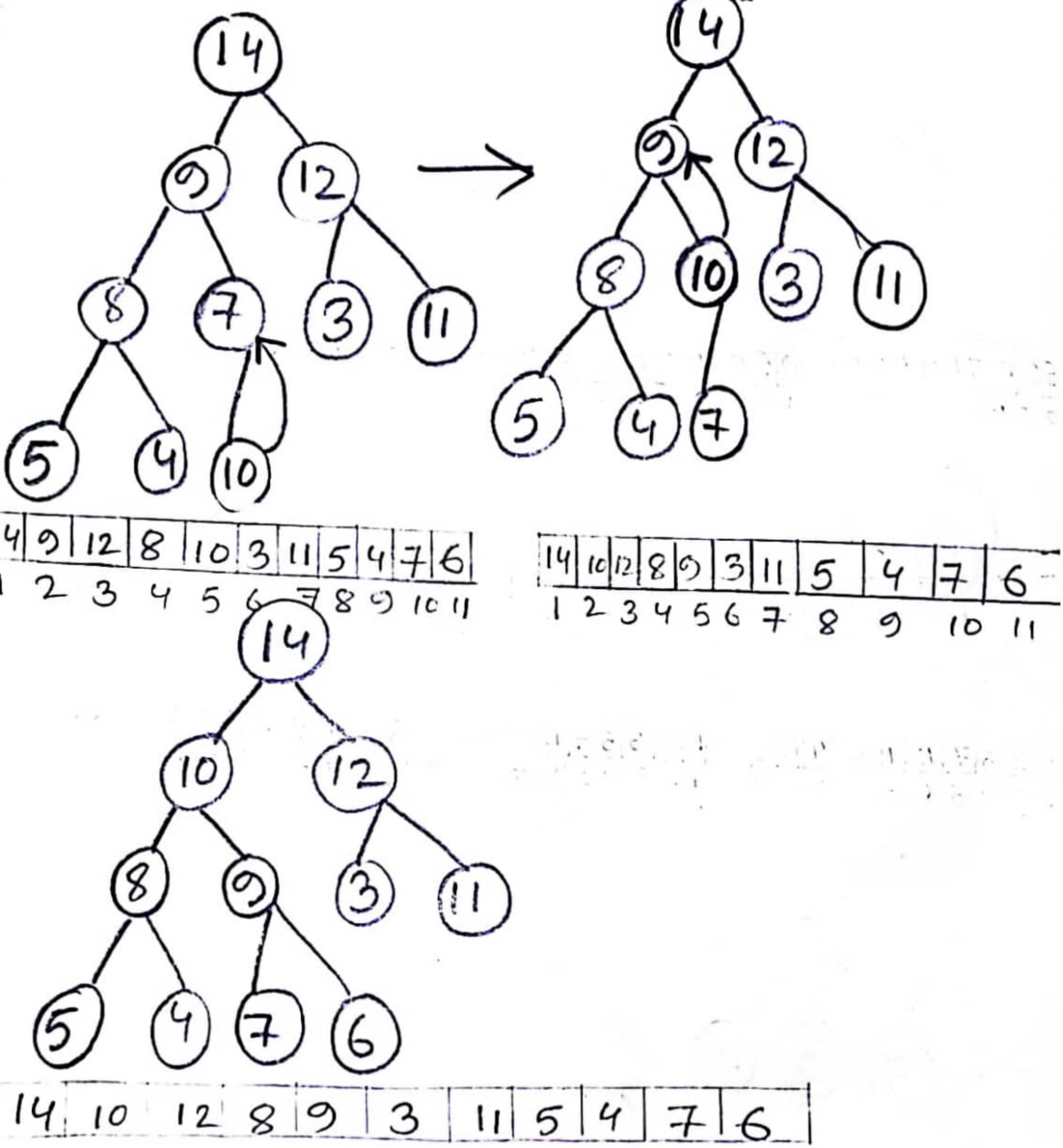
Or,

Max Heap (Parent Node > Child Node)







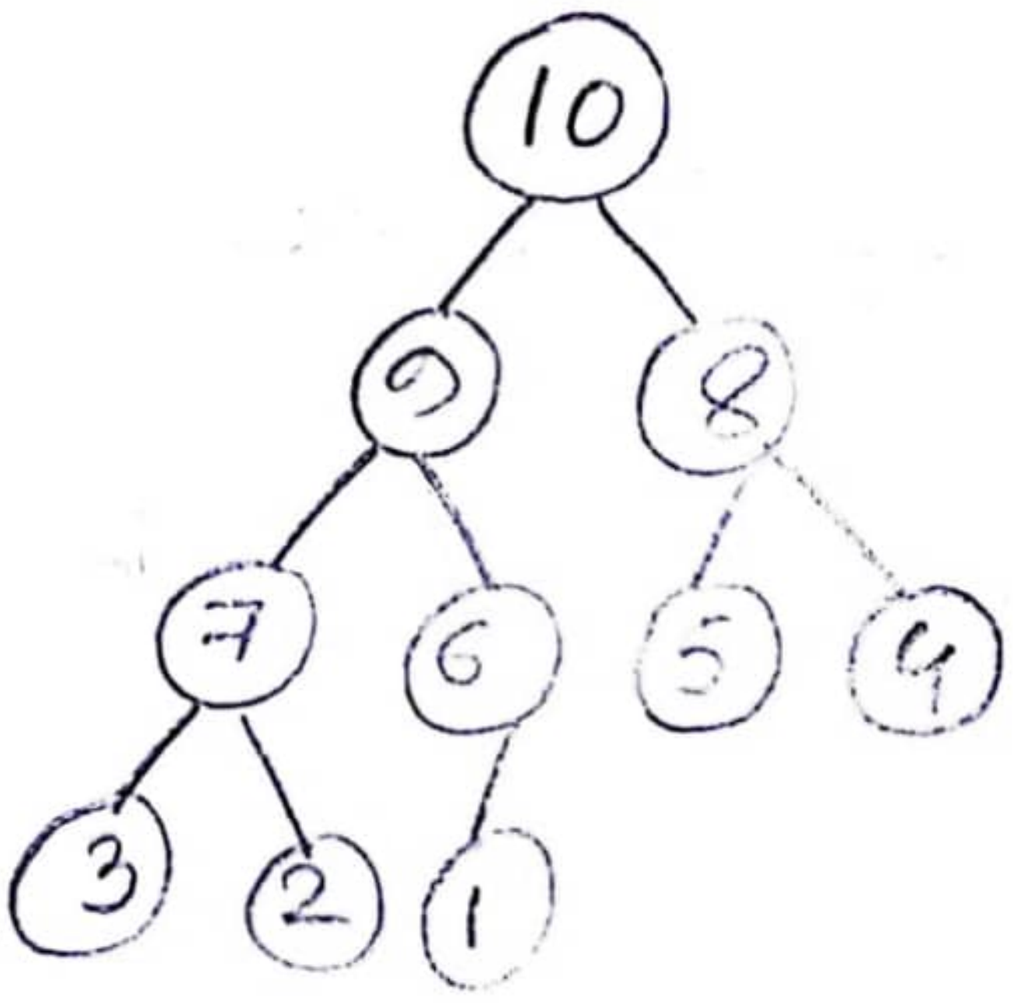

14
9
12
8
7
3
11
5
4
10
14
9
12
8
10
3
11
5
4
7
6
1 2 3 4 5 6 7 8 9 10 11
14
10
12
8
9
3
11
5
4
7
6
1 2 3 4 5 6 7 8 9 10 11
14
10
12
8
9
3
11
5
4
7
6
14 10 12 8 9 3 11 5 4 7 6

2(c) OR

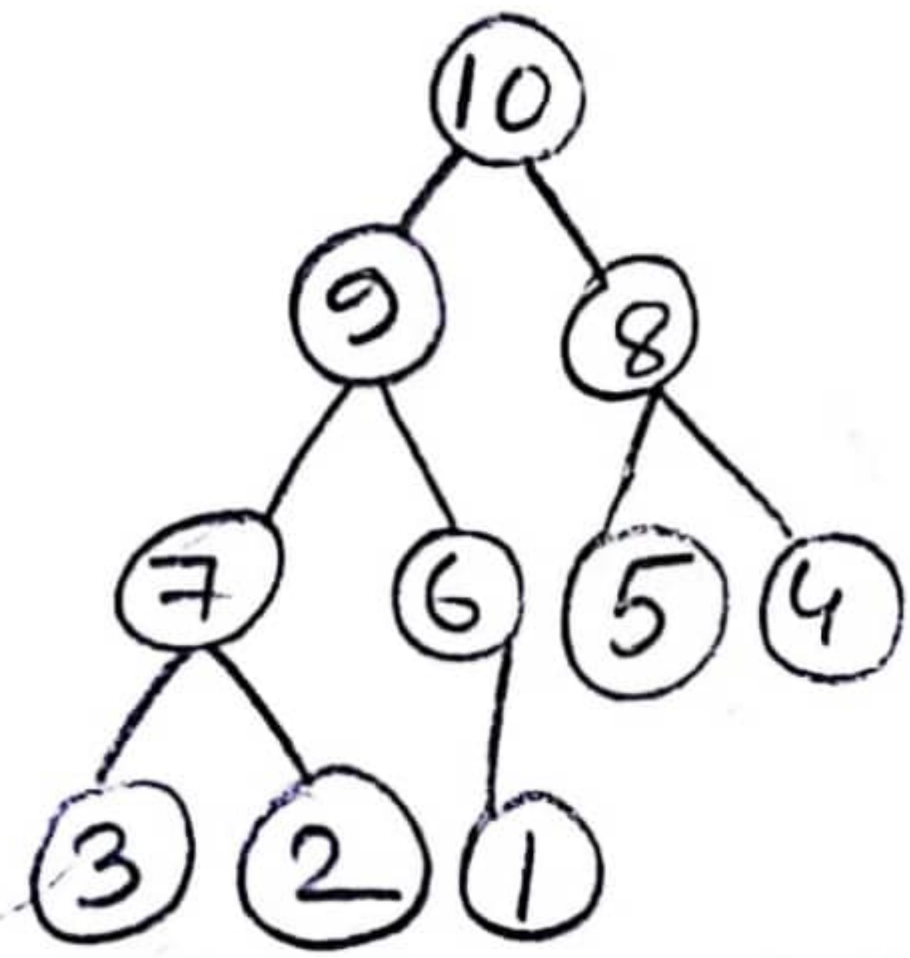
Show how can sort the numbers from the following Heap A. Illustrate each step.

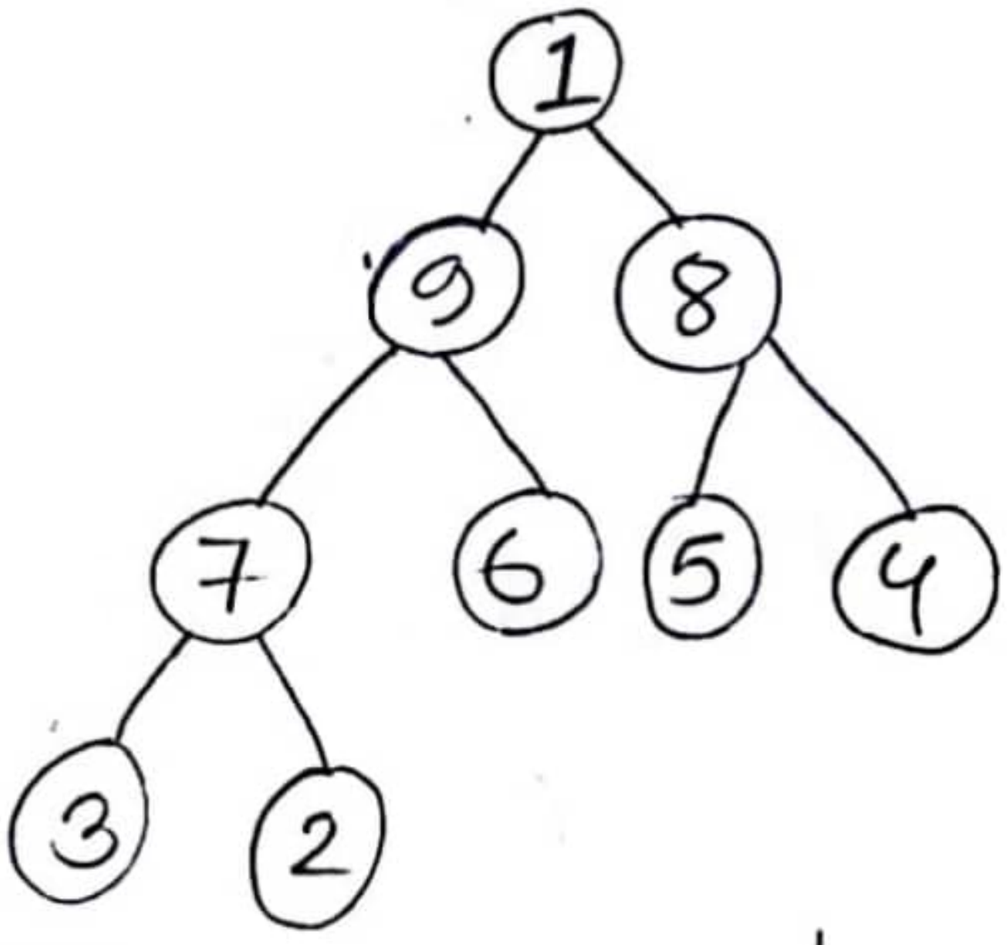
$A = \{10, 9, 8, 7, 6, 5, 4, 3, 2, 1\}$

Answer:



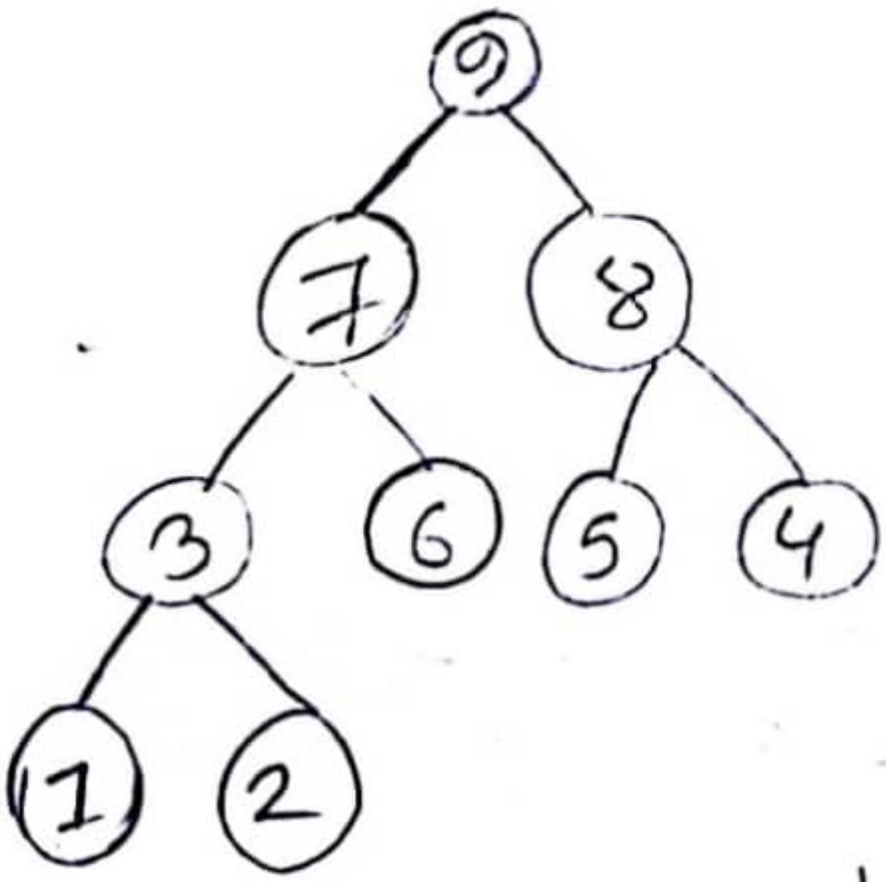
Deletion

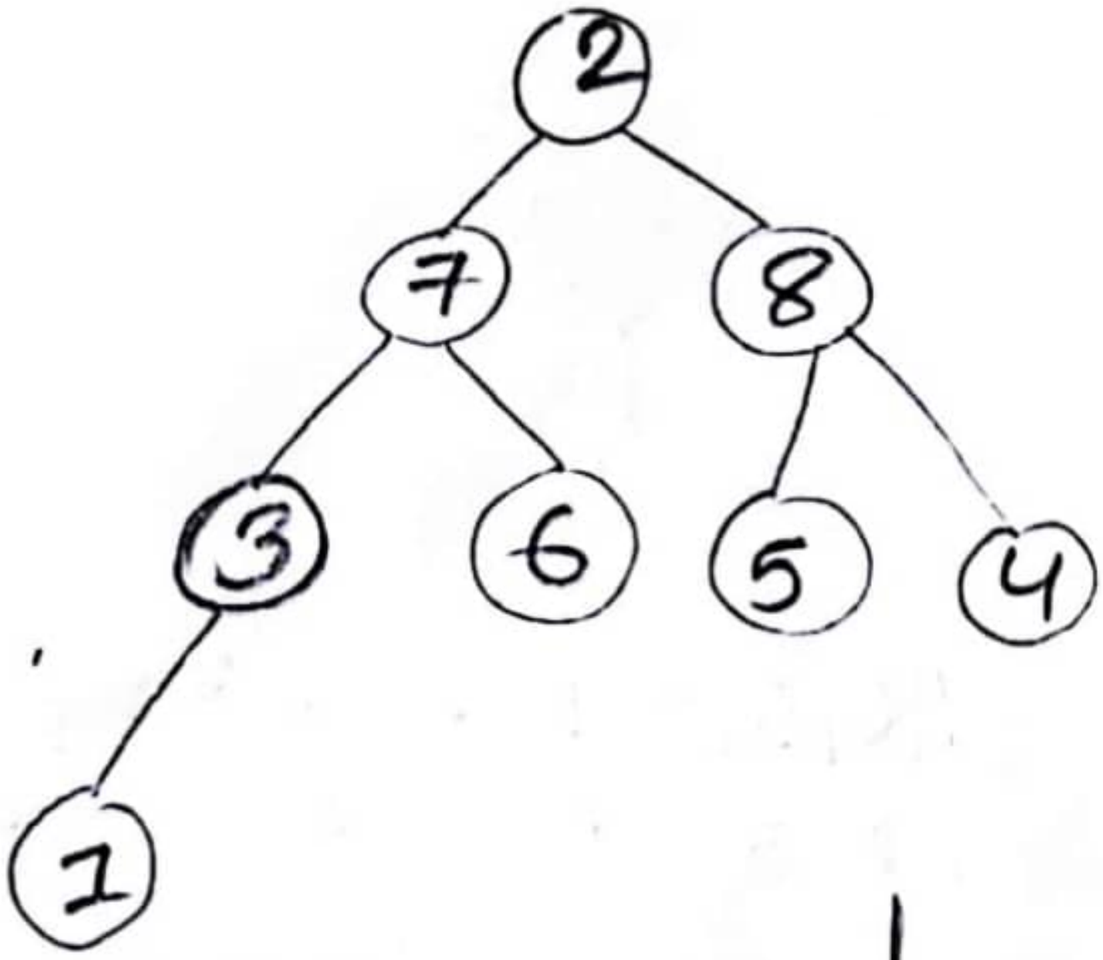




10	9	8	7	6	5	4	3	2	1
1	2	3	4	5	6	7	8	9	10

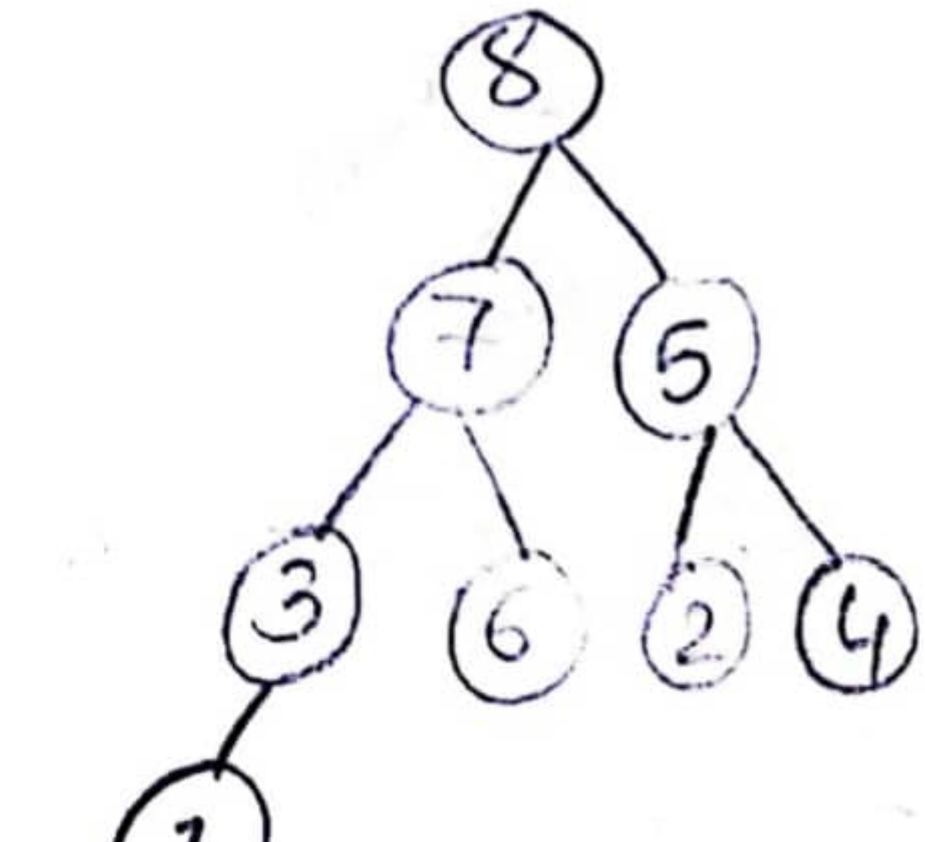
1	9	8	7	6	5	4	3	2	10
1	2	3	4	5	6	7	8	9	10

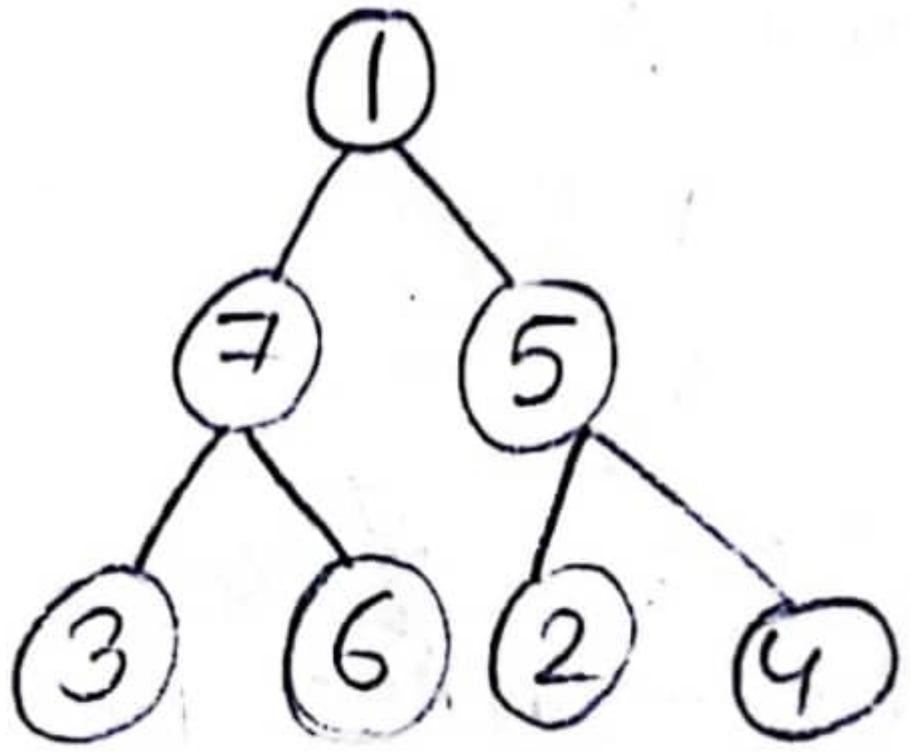




9	7	8	3	6	5	4	1	2	10
1	2	3	4	5	6	7	8	9	10

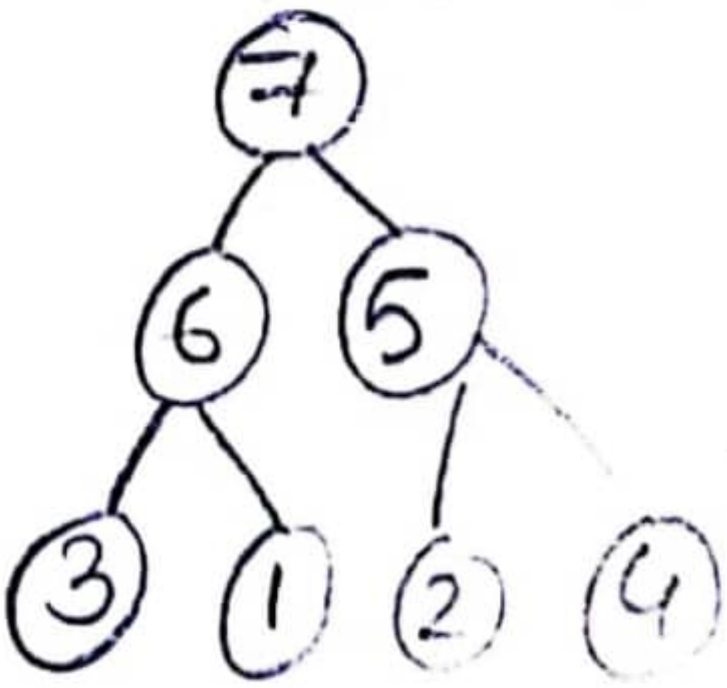
2	7	8	3	6	5	4	1	9	10
1	2	3	4	5	6	7	8	9	10

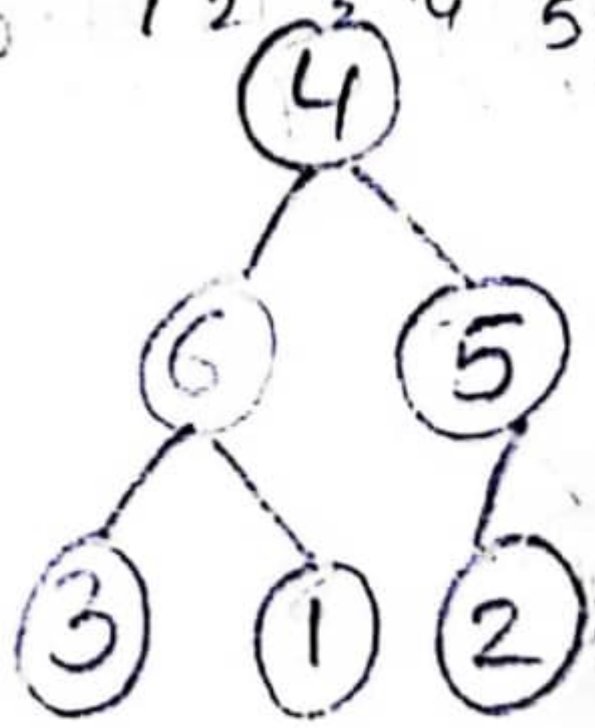




8	7	5	3	6	2	4	1	9	10
1	2	3	4	5	6	7	8	9	10

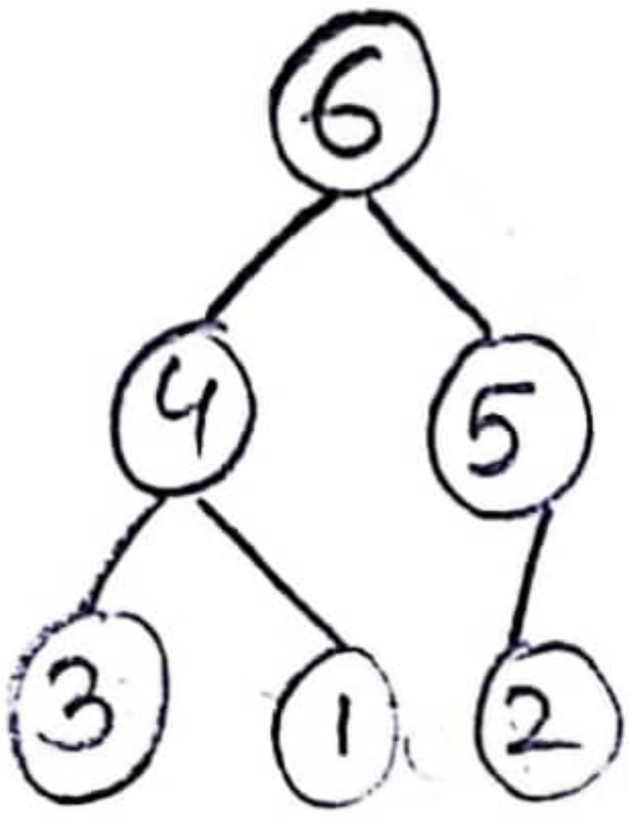
1	7	5	3	6	2	4	8	9	10
1	2	3	4	5	6	7	8	9	10



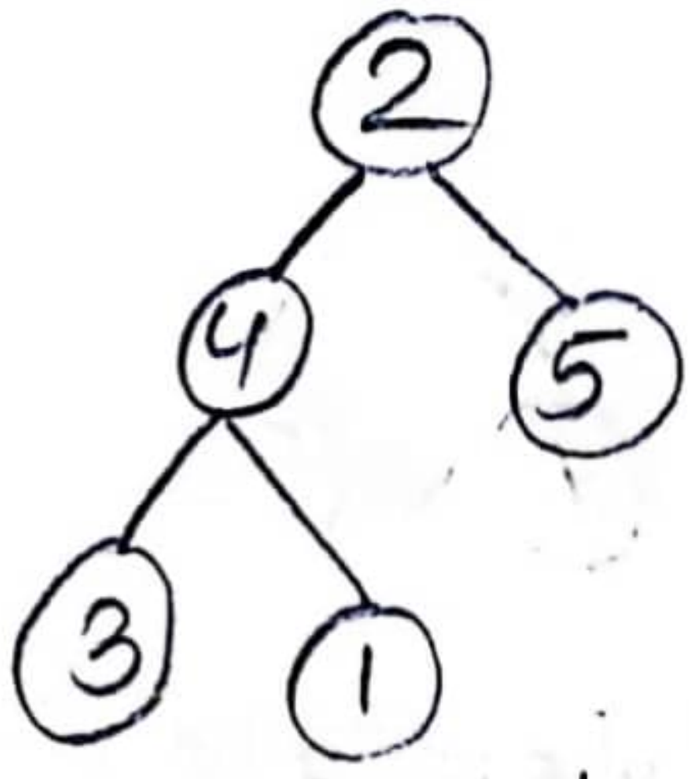


7	6	5	3	1	2	4	8	9	10
1	2	3	4	5	6	7	8	9	10

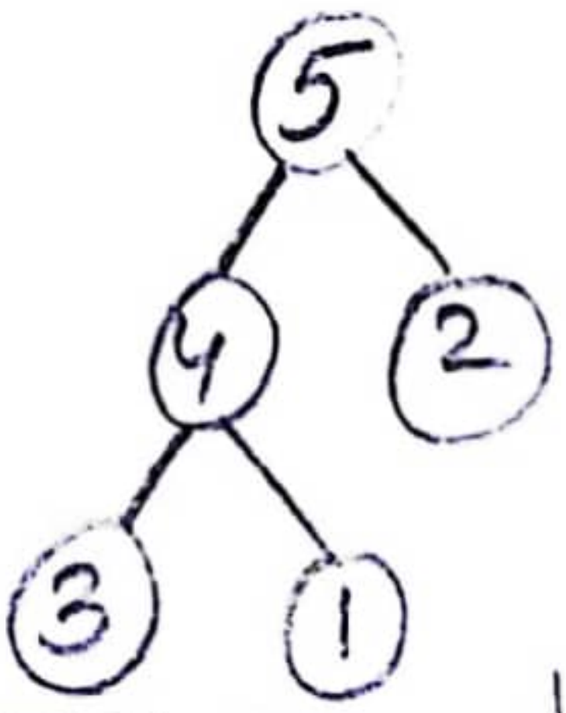
4	6	5	3	1	2	7	8	9	10
1	2	3	4	5	6	7	8	9	10



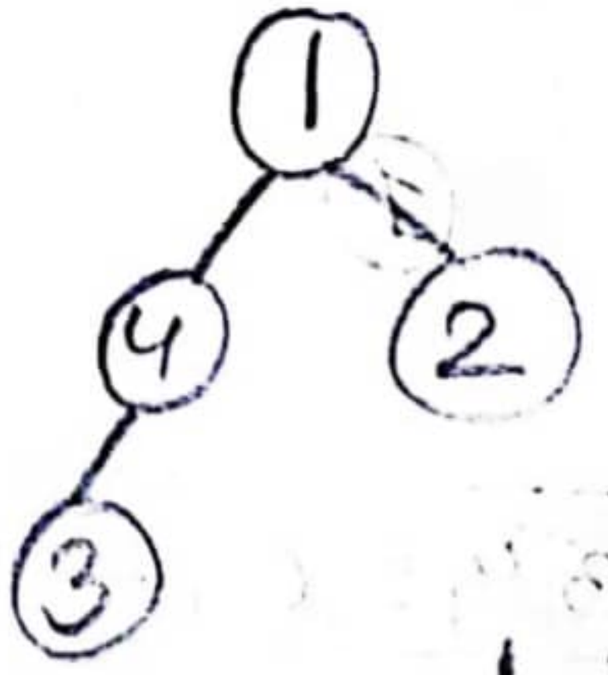
6	4	5	3	1	2	7	8	9	10
1	2	3	4	5	6	7	8	9	10



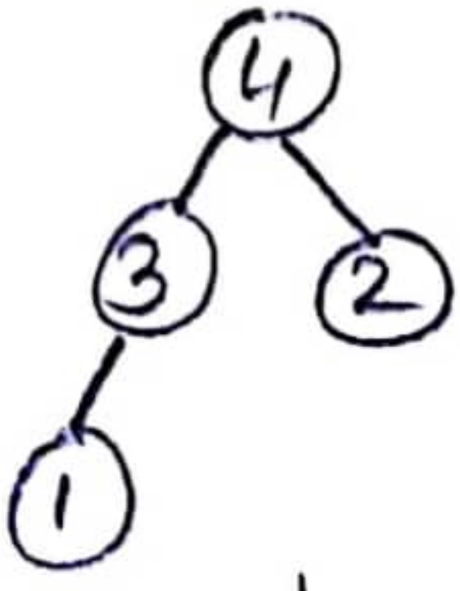
2	4	5	3	1	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



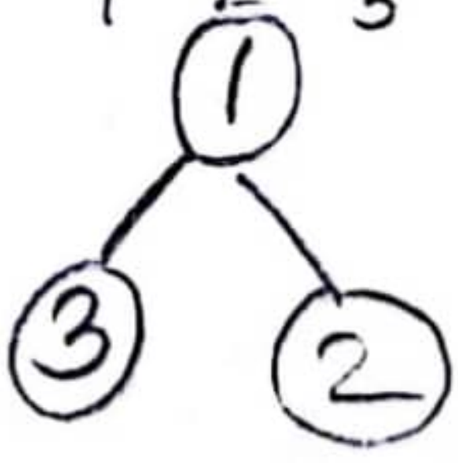
5	4	2	3	1	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



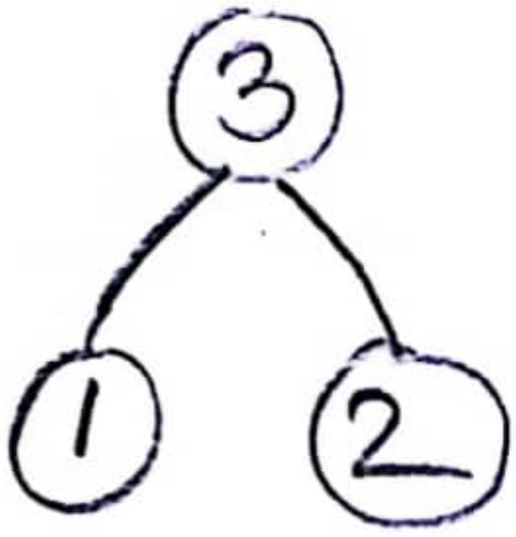
1	4	2	3	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



4	3	2	1	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



1	3	2	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10



3	1	2	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10

2	1	3	4	5	6	7	8	9	10
1	2	3	4	**5**	6	**7**	8	9	10

(1)

1	2	3	4	5	6	7	8	9	10
1	2	3	4	5	6	7	8	9	10

3(a)

To find the optimal parenthesization of a matrix chain product with dimensions (5,5,6,6,8) we can use the dynamic programming approach for matrix chain multiplication.

Here's the optimal parenthesization:

((M1*M2)* ((M3*M4)* M5))

Where M1 is a 5x5 matrix, M2 is a 5x6 matrix, M3 is a 6x6 matrix, M4 is a 6x8 matrix and M5 is an 8x1 matrix.

This parenthesization minimizes the total number of scalar multiplications required for the matrix chain product.

3(a) or,

To find the longest common subsequence (LCS) of the given sequence using dynamic programming, here are the steps:

① Create a table with dimensions (len(X)+1) x (len(Y)+1) to store the lengths of LCS of subproblems.

② Initialize the first row and first column of the table with zeros, as LCS of any sequence with an empty sequence is zero.

③ Iterate through each cell in the table, using the following rules: If X[i-1] == Y[j-1], then set the value in the current cell to the value in the diagonal cell (i-1, j-1)+1.

Otherwise, set the value in the current cell to the maximum of the value in the cell above (i-1, j) and the cell to the left (i, j-1).

④ After filling the entire table, the bottom-right cell (len(X), len(Y)) will contain the length of the LCS.

⑤ To find the LCS itself, start from the bottom-right cell and backtrack using the same rules to find the characters that contribute to the LCS.

Here's the table showing the lengths of LCS for each subproblem :-

$x = 0\,1\,1\,0\,1\,0\,1$

$y = 1\,0\,0\,0\,1\,1$

		1	0	0	0	1	1
	0	0	0	0	0	0	0
0	1	0	↖↑ 0	↖ 1	↖ 1	↖ 1	← 1	← 1
1	2	0	↖ 1	← 1	← 1	← 1	← 1	← 1 ↑
1	3	0	↖ 1	← 1 ↑	← 1	1 ↑	← 1	1 ↑
0	4	0	↑ 1	↖ 2	↖ 2	↖ 2	← 2	← 2
1	5	0	↖ 1	← 2 ↑	2 ↑	← 2	↖ 3	↖ 3
0	6	0	↑ 1 ↑	↖ 2	↖ 3	↖ 3	← 3	↑ 3
1	7	0	↑ 1	2 ↑	3 ↑	← 3	↖ 4	↖ 4

LCS - 1001

The length of the LCS is 4. To find the LCS itself, we can backtrack from the bottom-right cell:

(a) Start at cell (6,5) with value 4.

(b) Move diagonally left and up if the characters match, otherwise move either up or left, based on the greater value in the adjacent cells.

(c) Repeat ~~untill~~ until we reach cell (0,0)

By following this procedure, the LCS of the given sequences is: {1, 0, 0, 1}.

Spring 2023

3(b)

what is optimal substructure? Show that matrix-chain multiplication problem has this property.

Optimal substructure is a property in dynamic programming where the optimal solution to a larger problem can be constructed by utilizing optimal solution of its smaller sub-problem

let's demonstrate the optimal substructure property of the matrix chain multiplication problem using the example.

Consider the matrices A, B and C with dimensio

A = 10X30

B = 30X5

C = 5X60

To find the optimal way to multiply these matrices, we can break down problem into subproblems.

Let's say we have four matrices A, B, C and D with dimensions.

A = 10 X 30

B = 30 X 5

C = 5 X 60

D = 60 X 20

① (A(BC))D: Multiply A with the result of (BC) then multiply with D.

② A((BC)D): Multiply BC first, then multiply the result with A, and finally multiply the result with D.

~~subproblem~~

subproblem1: (A(BC)) optimal solution:

A(BC) requires 10 X 30 X 5 + 10 X 5 X 60 = 4500

Subproblem

Subproblem 2: (BC)D requires 30X5X60+

30X60X20 = 57000

By using optimal solutions of these sub-problems we find the optimal solution for entire problem.

① (A (BC)) D: 4500 + 57000 = 61500

Now,

Subproblem 3: A((BC)D) requires 10X30X60

+ 10X60X20 = 42000

Comparing these, we can see that A((BC)D) is more efficient

This example illustrates the optimal substructure ~~property~~ property in matrix-chain multipication. The optimal solution to the original problem (ABC)D can be constructed by combining the optimal solution of subproblem (A(BC))D and A((BC)D).

3(b) OR

What is overlapping subproblem?

⇒ An overlapping subproblem refers to a situation in dynamic programming where the some subproblem is sloved multiple times in a recursive algorithm. Show that the longest common subsequence problem has this property.

⇒ The LCS problem exhibits the overlapping subproblem property because when slouing it, you often encounter the same sub problem multiple times

Here's an example that illustrate the overlapping subproblem property of LCS

We have two strings "AGGTAB" and "GXTXAYB".

To find LCS between those two string we compare char from the end.

The last char are both B. So we have LCS of length 1.

(1) LCS ("AGGTAB", "GXTXAYB") = max (LCS ("AGGTAB", "GXTXAY"), LCS ("AGGTA", "GXTXAYB")).

So, let's focus ~~theas~~ these two subproblem,

⇒ LCS ("AGGTAB", "GXTXAY"): To slove this we compare last characters 'B' and 'Y' which are different. This leads two sub-problems:

a. LCS ("AGGTA", "GXTXAY")

b. LCS ("AGGTAB", "GXTXAYB")

⇒ LCS ("AGGTA", "GXTXAYB"): Now we again compare the last char 'A' and

'B' which are different. This leads two or more subproblems.

a) LCS ("AGGGT", "GIXTXAY")

b) LCS ("AGGGTA", "GIXTXAY")

~~As you can~~ As we can see we encounter the subproblem "LCS ("AGGGTA", "GIXTXAY")" twice. This where the overlapping subproblem comes into play. Instead of sloving it multiple times, we can store solution and reuse it.

3(c)

Given Array,

$A = \{6, 1, 8, 1, 6, 2, 6, 1, 8, 2, 7\}$

(I) Indexing the elements of the array,

A =	6	1	8	1	6	2	6	1	8	2	7
	0	1	2	3	4	5	6	7	8	9	10

(II) Counting the frequency of each element and putting them into the count array, C.

C =	0	0	2	0	3	0	3	1	2
	0	1	2	3	4	5	6	7	8

(III) Now, by doing cumulative sum, we get,

C =	0	0	2	2	5	5	8	9	11
	0	1	2	3	4	5	6	7	8

(-1) (-1) (-1) (-1) (-1) (-1) (-1) (-1) (-1) (-1) (-1)

(iv) The sorted Array be, B.

$\therefore$ B =

2	2	4	4	4	6	6	6	7	8	8
0	1	2	3	4	5	6	7	8	9	10